Inverted, Frozen, and Flapping: Four Infrastructure Notes from an Operational Marine Data Platform

Deepak Krishna
MarisAI
India
deepakkrishna2206@gmail.com

Abstract—Operational research software accumulates infrastructure decisions that look like defects until the measurement behind them is produced. We report four such decisions from one running marine-data platform, each re-verified against its primary source rather than an existing write-up. First, a backend forecasting engine is tracked in MLflow without importing MLflow — the training side writes plain JSON, the machine-learning side reads it — and the stated reason for a lighter tracking package (avoiding a pandas/pyarrow downgrade) is checked live against that package's own PyPI metadata: correct for the exact release pinned, already relaxed one release later. Second, pinning a snapshot date and freezing a disk cache's time-to-live take a 52-model, 1,248-fit reproducibility audit from roughly 700 seconds to roughly 0.8 seconds per variable — a claim whose model/fit counts are artifact-backed and whose speedup figure is not, and is graded accordingly. Third, Copernicus Marine's two zarr access services trade an 11.9× measured speed difference depending on request shape, and we correct an unsupported "hours" framing of that cost found in this project's own working notes. Fourth, an ERDDAP data server answers 404 while reloading a dataset it still serves, indistinguishable from a genuinely retired one, which led one real dataset to be treated as retired for months; we resolve a live disagreement between this project's own source-code comments and its documentation over the actual size of a related retry-policy cost, using the arithmetic each account carries internally to determine which figure the sources themselves support.

Index Terms—experiment tracking, reproducibility, MLflow, cloud-native array formats, zarr, ERDDAP, retry policy, research software engineering

I. Introduction

An operational research platform accumulates infrastructure decisions that look, from outside the codebase, like a mistake: a machine-learning experiment-tracking library that the code being tracked is forbidden to import; a cache deliberately configured to ignore its own expiry policy; two ways to fetch the same array that differ by an order of magnitude in speed depending on which one is called for a given request shape; a data server that answers "not found" for a dataset that is not, in fact, gone. Each of these reads as a bug, a redundancy, or an oversight until the specific measurement behind it is produced. This paper reports four such decisions from one running system — MarisAI, a marine ocean-data platform combining live provider ingestion, a per-site forecasting engine, and offline species-distribution and harmful-algal-bloom pipelines — each with the measurement that justified it, the mechanism that makes it necessary rather than merely defensible, and, in three of the four, a place where the justification itself has already partly gone stale: an upstream package's own version pin moved, a documentation figure disagrees with the code next to it, or a summary line's arithmetic does not say what it appears to say.

This is a short-form, practitioner-notes contribution in the SoftwareX tradition: four self-contained technical findings under one introduction, each independently useful to a reader who touches only one of the four sections. What ties them together is not a shared subsystem — one section is about a Python dependency graph, one about a disk cache, one about a cloud array format, one about an HTTP retry policy — but a shared discipline: every claim below was re-verified against a primary source this session rather than carried forward from an existing write-up, and every place that re-verification produced a different answer than the existing write-up is reported rather than silently corrected, following the same evidence-grading convention this project's other papers use.

A. Contributions

1) A dependency inverted rather than added (§II): the backend forecasting engine is tracked by MLflow without the backend importing MLflow, and the specific reason a lighter package is required (a pandas/pyarrow downgrade the full package would force) is checked live against the package's own published metadata rather than assumed — which shows the reason holds for the exact release pinned here, but has already been relaxed in the package's newer releases.
2) Two cache decisions that turn a 700-second-per-variable audit into 0.8 seconds (§III): pinning a snapshot date so a cache key is stable, and a frozen-cache override that is correct specifically because the window it serves cannot be revised upstream.
3) The same cloud array format, two chunkings, opposite winners (§IV): Copernicus Marine's two zarr services trade a 12-fold speed difference depending on whether a request is one snapshot of the whole globe or one bounded area across many timesteps, and a needed correction to an overstated claim about how bad the wrong choice actually is.

4) A 404 that is not proof a dataset is gone (§V): an ERDDAP data server answers "Currently unknown datasetID" while reloading a dataset it still serves, indistinguishable by status code from a genuinely retired one, with a documented case of a real dataset believed retired for months on that basis alone — and a second, independent inconsistency inside this project's own documentation about how expensive retrying the wrong kind of failure actually is.

II. An MLflow Dependency Inverted, Not Added

a) Claim: The backend forecasting engine's training runs are tracked in MLflow [1] without the backend ever importing MLflow. The training script writes an immutable, plain-JSON run record to backend/models/forecasting/_reports/runs/<timestamp>/, using only the standard library; a separate module on the machine-learning side, marine_ml.ingest_forecasting, reads those directories and logs them into MLflow after the fact. The tracked system is never aware that tracking exists.

b) Why the direction is inverted: The natural design — log directly from the training script — would put an MLflow import inside backend/, which this codebase treats as a hard boundary: the API's import graph is kept free of the modelling stack for the same reason services/predictions.py serves precomputed grids rather than running inference in-process. A tracking client imported by a training script still lands in that tree. Inverting the dependency — write JSON the training script already had to write; read it from the side where MLflow is actually installed — keeps the boundary intact while still getting every run permanently recorded, which a fixed filename previously did not: two training batches run back to back had silently overwritten each other's report, the concrete failure that motivated this design. backend/tests/test_training_run_record.py enforces both properties directly: one test walks every .py file under backend/ for an import mlflow or from mlflow line, and another checks neither pyproject.toml nor requirements.txt mentions mlflow at all. Both were re-run for this paper and pass, with 0 offending imports found.

c) Why mlflow-skinny, checked rather than assumed: The tracking side pins mlflow-skinny==3.6.0, not plain mlflow, alongside pandas==3.0.5 and pyarrow==25.0.0. The stated reason is that the full package pins an older pandas and pyarrow ceiling that would force a major-version downgrade underneath a multi-million-row parquet feature store. That claim is not taken on the repository's word here: PyPI's own release metadata for mlflow==3.6.0 — the release matching the mlflow-skinny version actually pinned — was read directly, and it does require pandas<3 and pyarrow<23, confirming the stated reason exactly. But the same lookup against mlflow's current latest release, 3.16.0, shows that ceiling has already moved: pandas<4 and pyarrow<26, both of which 3.0.5/25.0.0 already satisfy without any downgrade. The inversion's justification is therefore correct as documented, but it is a fact about one pinned release rather than a stable fact about "mlflow" — worth stating with a version number attached, which neither this repository's own comments nor research.md's write-up of them originally did, and worth re-checking at the next dependency bump rather than treated as permanent.

d) Tracking never fails a training run: A harmful-algal-bloom training run takes roughly 40 minutes; losing one to a locked SQLite store or a NaN metric would be a strictly worse outcome than an untracked run. Every logging call in marine_ml.tracking is wrapped in a context manager that catches any exception and downgrades it to a warning, and the run-opening code itself falls back to an inert, no-op handle if MLflow is not importable or the tracking store cannot be reached at all — the training pipeline proceeds identically either way, just without a recorded run.

e) Why it matters: A dependency that is added in the conventional direction (import the tracker where the work happens) is the version that gets built first and is the version that erodes an import-graph boundary a team specifically drew for a different reason. Inverting it costs one extra module and one regression test, and the version-checked justification above is the kind of claim that ages — a training script's dependency choices are usually re-examined only when something breaks, and this one's stated reason had already partly stopped being true one release upstream by the time this paper checked it.

III. Reproducing a 52-Model Audit in Under a Second per Variable

a) Claim: A companion study (Rising Skill, Falling Skill, research/README.md) trains and scores 52 models (13 variables × 4 forecast horizons) against two baselines, then evaluates each with a leave-one-site-out transfer test across 24 sites, for 1,248 held-out fits in total (52×24, not a third factor folded into the 52 — a compressed line in the study's own README reads as one product of three numbers when it is actually two separate multiplications, one for model count and one for held-out fit count). The main experiment runs end to end in 5229.8 seconds and a companion seasonality ablation in 2411.5 seconds, both read directly from the committed run-metadata files this session, entirely offline.

b) The mechanism: Two independent decisions make that possible. First, the training window's as_of snapshot date is pinned (2026-08-10), rather than computed from "now" as a live training run would, so every cached provider request the audit makes has a stable cache key across reruns; without it, each rerun would recompute a slightly different window and re-request roughly two years of data per variable per site from scratch. Second, an environment variable (MARISAI_FROZEN_HISTORY_CACHE=1) tells the

disk cache to serve an entry regardless of its normal 6-hour time-to-live. That TTL exists because a live request asks for a window ending "now," and yesterday's answer to that question is stale by construction; a pinned, closed historical window is the opposite case — it cannot be revised by the upstream provider after the fact, so re-fetching it would yield the same bytes at real cost and occasional risk (a flapping provider can silently return a slightly shorter series on a rerun, making two runs of the same experiment incomparable). The override is set only by the three experiment scripts that need it and is read nowhere on a live serving path, so a request for "today's" data can never accidentally inherit a frozen answer.

c) What is, and is not, a stored measurement: The audit's scale — 52 models, 1,248 fits, and the two total run durations above — is read directly from committed report files (research/data/results/meta.json, loso.json, ablation_meta.json) and is reproducible by re-running the committed pipeline. The audit's speedup, by contrast — from roughly 700 seconds to roughly 0.8 seconds per variable, an 875× factor — is stated identically in three places (the study's README, the caching function's own docstring, and the experiment script's module docstring) but traces to no committed benchmark log or named timing script in any of the three. It is reported here because it is the number that explains why a 52-model audit plus 1,248 held-out fits was practical to run at all, but it carries a different evidence weight than the counts beside it, and is graded accordingly rather than folded into the same claim.

d) Why it matters: A cache that ignores its own TTL is, by default, exactly the kind of decision a later reviewer flags as a bug: caches are supposed to expire. The distinction that makes it correct here — a pinned historical window is a different object than a live one, and deserves a different caching policy — is not visible from the code alone; it is visible only once the access pattern (an offline reproducibility experiment, not a live dashboard request) is stated explicitly, which is why the override is scoped to the three scripts that state it rather than made a global default.

IV. Access-Pattern-Dependent Performance in ARCO Zarr Stores

a) Claim: Copernicus Marine publishes the same physics and biogeochemistry products through two Analysis-Ready Cloud-Optimized (ARCO) zarr services — part of a broader shift toward cloud-native, chunked array formats for large scientific datasets [2] — with opposite chunking layouts: arco-geo-series stores huge latitude/longitude chunks with one timestep each, and arco-time-series stores fine latitude/longitude chunks with huge time chunks. Which one is faster depends entirely on the shape of the request, and this one codebase makes requests of both shapes: a whole-globe, single-timestep snapshot for map-tile rendering and the global habitat-model build, and a bounded-area, many-timestep series for the regional downloader and the per-site forecasting engine. Using the wrong service for a given request shape is a correctness-preserving but severe performance regression, not a crash, which is exactly the kind of defect that survives unnoticed until someone times it.

b) The one direction actually measured: docs/ml-notes.md records a direct comparison, dated 2026-08-10, for one year of global monthly physics coarsened to 0.25°: arco-geo-series completes in 7.5 seconds against arco-time-series's 89.0 seconds for the identical request — an 11.9× difference for picking the service built for the opposite access pattern. The reversal for the opposite request shape (a bounded area, many timesteps, where arco-time-series's fine spatial chunking and coarse time chunking should win) is not a separately timed measurement in any source read for this paper; it is stated as a direct, mechanistic consequence of the two services' documented, opposite chunk layouts, and is treated here as asserted rather than measured. Three access patterns exist in this one codebase (map-tile snapshot, regional download, whole-globe forecast grid), and the service is selected per call rather than as a module-wide default specifically so a future caller cannot silently inherit the wrong one for its own request shape.

c) A second, independent hazard: a missing depth bound: The 50-level _my_ reanalysis products carry every one of those levels even when only the surface is needed. Passing minimum_depth/maximum_depth into open_dataset so the server subsets by depth as well as by space and time turns a request that otherwise did not finish in 15 minutes into one that completes in roughly 60 seconds. This is a different mechanism from the service-selection finding above — a missing server-side filter, not a choice between two services with different chunking — and is kept as a separate claim rather than merged into the same headline number.

d) A claim this paper does not repeat: Both this project's own working notes (CLAUDE.md) and research.md's write-up of them describe picking the wrong ARCO service as turning "a ∼10-second fetch into minutes/hours" (CLAUDE.md) or "hours" outright (research.md). Neither figure is supported by any measurement located this session: the one timed comparison above is 7.5s against 89.0s, an 11.9× factor and not hours, and the depth-bound hazard's worst measured case is stated as "did not finish in 15 minutes," which bounds how long an engineer waited before giving up, not how long the request would actually have taken. This paper reports only the two measured-or-explicitly-stated pairs above rather than the unquantified "hours" framing.

e) Why it matters: A cloud-native array format advertised as one product can hide two internally inconsistent access patterns behind one API surface, and the two can differ by an order of magnitude for the same logical request. The fix costs one keyword argument per call site;

the hazard is that the module-level default silently applies to every future caller unless the choice is forced at each call site, which is why this codebase treats it that way.

V. ERDDAP Answers 404 While Reloading a Dataset

a) The corrected belief: For months, NOAA CoastWatch's ERDDAP-served GEBCO_2020 bathymetry dataset was treated in this codebase as permanently retired, because every request for it returned HTTP 404. That belief was wrong. Re-probed 2026-08-05, GEBCO_2020 returns 200 with correct depths. What actually happens, confirmed by the fix committed that same day, is that the whole server flaps: over one probing session it went fully 503, then served GEBCO_2020 at 200 for minutes, then answered 404 for it again for roughly 100 seconds before recovering. ERDDAP unloads a dataset while reloading it and answers 404 "Currently unknown datasetID" during that window — indistinguishably, by status code and message, from a dataset that has actually been removed. The production bathymetry provider now reads gebco2021 from a different host (Ifremer's ERDDAP), chosen because it is the newer GEBCO release and happened to stay up throughout the flapping session, not because CoastWatch's copy is actually gone.

b) The retry policy this forces: A 404 cannot be treated as either always permanent or always transient without cost on one side or the other, so the classification is asymmetric and specific to the status code. Confirmed directly against backend/forecasting/history.py's retry classifier and its currently-passing test suite: a 404 is retried exactly 2 times, with 2.0 seconds of backoff, because a request landing inside a reload window recovers on the second attempt; a 400 (and the other permanent 4xx codes, 403/410/422) gets 1 attempt and no retry at all, because those mean the request itself is malformed and will stay malformed; and a 503 (with the other transient codes, 408/425/429, and any 5xx) is retried up to 3 times with 8.0 seconds of total backoff, because a fully overloaded or restarting server is exactly the case unlimited-within-budget retrying exists for.

c) Both costs, measured: Getting this wrong costs real time in both directions, and both directions are documented with a specific number rather than left as a general concern. Retrying a permanent failure with the old, uniform three-attempt policy added the full 8-second backoff (2.0s + 6.0s) to every request against that dataset. The two sources closest to the implementation — history.py's own code comment and the regression test built specifically around this scenario — both state this turned a warm 6-second response into 14.5 seconds; this project's own documentation page and research.md's write-up of it instead state a warm 2.2-second response becoming the same 14.5 seconds. Both pair with the identical 8-second penalty, and only one addition is close to the stated total: $6 + 8 = 14$, against $2.2 + 8 = 10.2$ — neither reaches 14.5 exactly, but the code-and-test figure is the one the arithmetic actually supports, and it is reported here as primary for that reason, with both figures stated rather than the disagreement silently resolved. In the other direction, retrying too conservatively costs geography rather than latency: a single transient ERDDAP 503 during one training run cost 6 of 24 training points — a quarter of that model's spatial coverage, including the entire Gulf Stream and Coral Sea — from a provider that supplies one constant column and had recovered again within seconds.

d) A live loose end: One committed test's own prose has not caught up with the correction above. backend/tests/test_download_gebco.py still narrates GEBCO_2020 as a dataset "the server retired" that "is gone," language describing the pre-2026-08-05 belief rather than the flapping-server explanation recorded in the provider module's own comment. The test itself is not wrong — it still correctly asserts the provider requests Ifremer's gebco2021 rather than GEBCO_2020, which remains true for the unrelated reason that Ifremer stayed available throughout the flap — but its docstring was never updated to match the correction shipped elsewhere in the same area of the codebase, a small, live instance of exactly the documentation drift this section's other paragraph reports.

e) Why it matters: A 404 is conventionally treated as the internet's most unambiguous status code: the thing you asked for does not exist. On an ERDDAP host mid-reload it means something closer to "ask again in a moment," and the two cannot be told apart from the response alone. Getting the retry policy wrong in either direction has a real, opposite cost — a slow page on one side, missing training geography on the other — and both costs were only visible once someone measured them rather than picked a retry count that felt reasonable.

VI. Discussion

Read together, the four findings share a shape that is easy to miss reading them one at a time: in every case, the decision that looks wrong from the code alone is the one a specific, dated measurement made necessary, and in three of the four, some part of the write-up describing that measurement has already drifted from the primary source by the time this paper re-checked it. The MLflow inversion's stated justification is exactly correct for the release pinned, and already one release out of date for the package in general. The 52-model audit's headline speedup is real but is prose, not an artifact, sitting next to counts that are. Two independent sources inside this same codebase — a source-code comment and its own regression test on one side, a documentation page and research.md's copy of it on the other — disagree by a factor of nearly three on how expensive a bad retry policy actually was, and the arithmetic embedded in the claim itself is what settles which one to believe rather than either source's authority. None of these is the kind of drift that changes

TABLE I

The four findings, what each decision looks like without its measurement, and this paper's evidence grade for the headline numbers (see shared/generated/macros.tex for the full per-group grading). Artifact = a committed report/data file or a currently-passing test reproduces the number on demand; Documentation = descriptive, reproducible prose with no stored benchmark output; Dated = a single, named-date measurement with no committed artifact; External = checked live against a third party's own published metadata.

§	Looks like	What was measured	Evidence
4.1	an unnecessary extra package (mlflow-skinny instead of mlflow)	keeps 3.0.5/25.0.0; full mlflow 3.6.0 pins pandas<3/pyarrow<23 (already pandas<4/pyarrow<26 in 3.16.0)	Artifact + External
4.2	a cache that ignores its own expiry	52 models, 1,248 fits (counts); 700s → 0.8s per variable (speedup)	Artifact (counts) + Documentation (speedup)
4.3	an arbitrary choice between two equivalent zarr services	7.5s vs 89.0s, one direction timed; depth bound 15min → 60s	Dated
4.4	an over-cautious retry count for a simple 404	2 attempts/2.0s (404), 1 attempt (400), 3 attempts/8.0s (503); 6s → 14.5s misrouted	Artifact (policy) + Documentation (cost)

the underlying finding; each is the kind that changes a specific number a reader might otherwise quote with more confidence than it has earned.

That pattern is not unique to infrastructure code, and this project's adjacent work on silent, plausible failure modes in this same platform's modelling pipelines found the identical structure at the modelling layer: a number that is real, computed correctly, and licensed by convention or by a prior write-up to mean more than a second check shows it to mean. The version-checked dependency claim, the counts-versus-prose distinction in the cache speedup, and the 6-versus-2.2-second reconciliation in §V are three small instances of exactly that discipline applied to this paper's own sources rather than to the system being described — which is the point: a paper reporting that infrastructure decisions need re-verification against their primary sources should not itself rest on an unverified secondary one.

VII. Conclusion

Four infrastructure decisions from one operational marine-data platform — an inverted MLflow dependency, a cache that deliberately ignores its own expiry, a per-call choice between two zarr services with opposite chunking, and an asymmetric retry policy for one ambiguous HTTP status code — each looks like a defect or an arbitrary choice until the measurement behind it is produced, and each measurement is reported here with the evidence grade it actually earns rather than a uniform confidence. Three of the four carried a documentation figure that had already drifted from its own primary source by the time this paper checked it: an upstream package's version pin that has since been relaxed, a per-variable speedup that is prose rather than a stored benchmark, and a retry-cost example that disagrees with itself by nearly a factor of three depending on which committed file is read. None of the four findings changes as a result; every one of the three corrections is reported explicitly rather than folded quietly into a rounder number, because a paper about verifying infrastructure decisions against their sources has to hold its own numbers to the same standard.

Reproducibility

Every number in this paper traces to a file, currently-passing test, git commit, or live third-party metadata lookup named in shared/generated/macros.tex, grouped and evidence-graded by section. Section II's claims are read from machine_learning/marine_ml/tracking.py, machine_learning/requirements.txt, and backend/tests/test_training_run_record.py (re-run for this paper: 10 passed), cross-checked against PyPI's release-specific JSON for mlflow==3.6.0 and mlflow==3.16.0. Section III's counts are read from research/data/results/{meta,loso,ablation_meta}.json and research/data/sites.csv; its mechanism from backend/forecasting/history.py and backend/scripts/run_paper_experiments.py. Section IV's numbers are read from docs/ml-notes.md. Section V's policy and its regression coverage are read from backend/forecasting/history.py and backend/tests/test_forecasting.py (re-run for this paper: 8 passed of the retry-classification tests), its historical correction from git commit 50438f0 and backend/services/download/providers/gebco.py, and its documentation inconsistency from docs/features/forecasting.md against those same two files. Every number in this paper was verified against those sources as of 2026-09-10.

References

[1] A. Chen, A. Chow, A. Davidson, A. DCunha, A. Ghodsi, S. A. Hong, A. Konwinski, C. Mewald, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, A. Singh, F. Xie, M. Zaharia, R. Zang, J. Zheng, and C. Zumar, "Developments in MLflow: A system to accelerate the machine learning lifecycle," in Proceedings of the Fourth International Workshop on Data Management for End-to-End Machine Learning (DEEM'20), 2020, pp. 1–4.

[2] R. P. Abernathey, T. Augspurger, A. Banihirwe, C. C. Blackmon-Luca, T. J. Crone, C. L. Gentemann, J. J. Hamman, N. Henderson, C. Lepore, T. A. McCaie, N. H. Robinson, and R. P. Signell, "Cloud-native repositories for big scientific data," Computing in Science & Engineering, vol. 23, no. 2, pp. 26–35, 2021.